

DESIGN SEARCH SYSTEMS

- DECIDING REQUIREMENTS
 - 1.Functional Requirements
 - 2.Non Functional Requirements
- CAPACITY ESTIMATION
 - 3.DAU-MAU
 - 4.Throughput
 - 5.Storage
 - 6.Memory
 - 7.Network and Bandwidth Estimation
- API DESIGN
 - 8.API Design:Search
- HIGH LEVEL DESIGN
 - 9.HIGH LEVEL DESIGN:Search Suboptimal
 - 10.HIGH LEVEL DESIGN:Search Indexing
 - 11.HIGH LEVEL DESIGN:How to support Fuzzy Search
 - 12.HIGH LEVEL DESIGN:Ranking
 - 13.HIGH LEVEL DESIGN:Search Final Design
- DEEP DIVE INSIGHTS
 - 14.Deep Dive Insights:Database Selection
 - 15.Deep Dive Insights:Data Modeling
 - 16.Deep Dive Insights:Into Query Correction Service

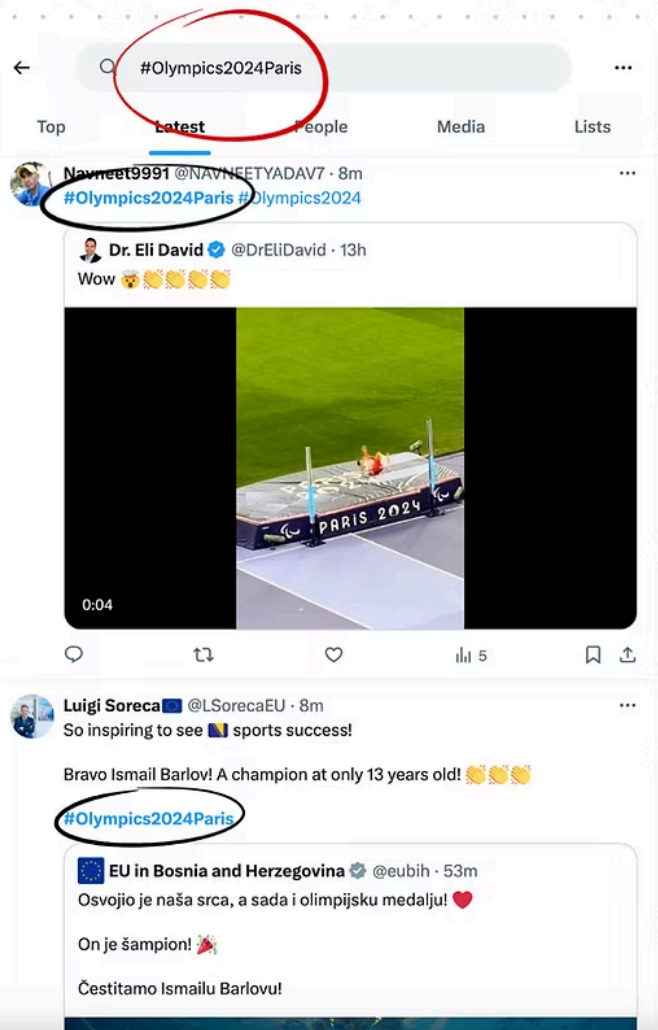
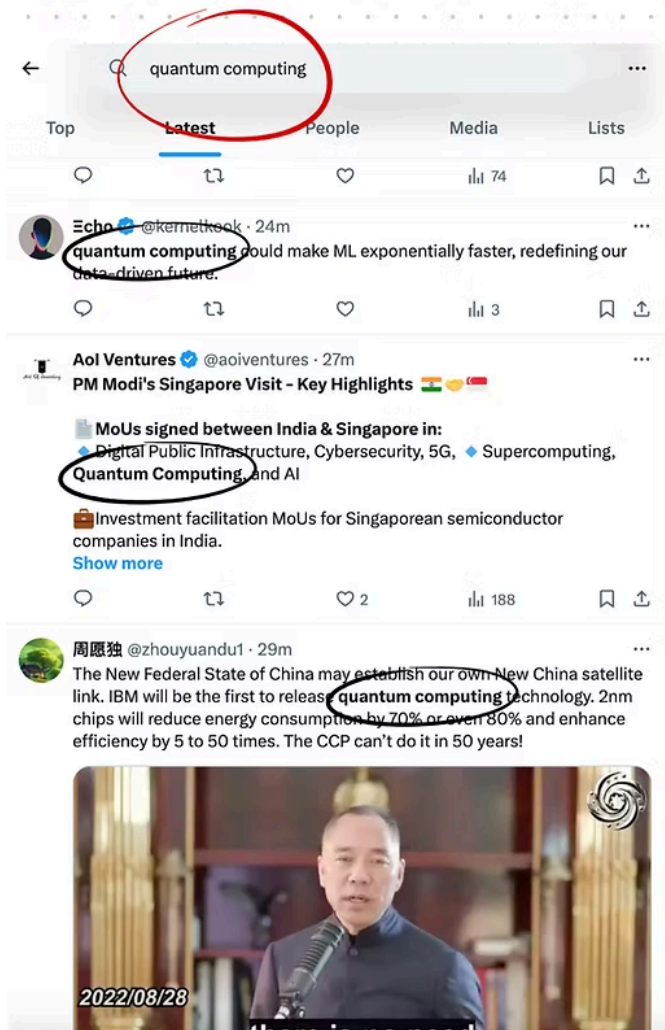
DECIDING REQUIREMENTS

Functional Requirements

Requirement	Description
Search	Users can search for tweets using keywords, hashtags, or usernames. The search will display all related tweets from newest to oldest.
Fuzzy Search	Our system should also support Fuzzy Search, which helps users find results even if they misspell words. For example, searching for "facebok" might still show results for "Facebook". This makes searching easier and more forgiving.

Examples:

- Suppose a user wants to search for posts related to "quantum computing". If they type this and search, they will see all the posts that mention "quantum computing".
- A user searches for #Olympics2024Paris. They will see all the posts where this hashtag is mentioned.
- If a user misspells and searches for "quantom computing", they will still see the correct results for "quantum computing".



User misspells and searches for "quantom computing" instead. Still he sees the correct results for quantum computing.



quantom computing



Top

Latest

People

Media

Lists



The Wolf of Defi ⚡️🔵 @thewolfofdefi · 7m



The trigger for #AI projects to start rallying was the launch of ChatGPT.

I expect something similar for **quantum computing**. Once we have a real-world breakthrough in **quantum computing**, which is inevitable, projects in this narrative will rally.

[\\$DNX](#) is the obvious leader. I'm
[Show more](#)



Dynex and 7 others



1



2



165



kinderspiel @duckundead · 7m



[\\$GOOGL](#) Google & [\\$IBM](#) IBM are working in areas related to AI & **quantum computing**. While they compete in some areas, they also collaborate on advancing technological research.



10



SuVa Protocol @suvaprotocol · 10m



Replying to @suvaprotocol

3/5

Quantum computing's unique ability to process massive amounts of data simultaneously will destroy current encryption methods. 🧠 Once quantum supremacy is reached, your digital assets could be exposed in an instant.
[#DigitalAssets](#) [#Cybersecurity](#) [#Blockchain](#)

Non Functional Requirements

Requirement	Description
Availability	Our system should be highly available 99.9999% of the time. With millions of users searching simultaneously, it's critical that the search function never goes down.
Low Latency	Search results must load quickly. Any noticeable delay could lead to user frustration.
Scalability	The user base of the platform is expected to grow with time. The system should be able to scale up in the future smoothly.

CAPACITY ESTIMATION

DAU MAU

Metric	Value
Daily Active Users (DAU)	100 million
Monthly Active Users (MAU)	400 million

Throughput

Since we are designing the search functionality on Twitter, we will calculate the throughput only for search operations. Every search operation reads some information (searches) from our system.

Reads

There is only one primary way to read data from our system:

- **Searching posts by keywords**

Let's assume each user searches **5 times a day**.

Total searches per day:

DAU $\times$ 5 searches per day

= 100 Million $\times$ 5

= 500 Million searches per day

Searches per second: = 500 Million searches / (24 $\times$ 60 $\times$ 60) seconds per day $\approx$ 5,787 searches per second

Storage

Designing Twitter Search Functionality

We are designing a search functionality for Twitter, but what data will it search? It will search tweet data. Let's look at how much storage these tweets take up.

Note: For simplicity, let's assume we're only dealing with text tweets since our search feature performs text searches anyway.

Estimating Storage Requirements

We know we have **100 million daily active users**.

- Suppose each person tweets, on average, **once a day**.
- Let's assume each tweet has about **200 characters** on average, which roughly translates to **200 bytes** (considering 1 byte per character).

Storage Calculation Per Day:

Total storage per day

= 100 million users $\times$ 1 tweet per day $\times$ 200 bytes per tweet

= 20 billion bytes/day = 20 GB/day.

Total storage in 10 years

= 20 GB/day × 365 days/year × 10 years = 73,000 GB = **73 TB**.

Memory

By memory, we mean cache memory size.

Accessing data from a database takes a long time, but if we want to access it faster, we use cache.

Let's assume that we store **5%** of the most frequently accessed tweets data in the cache.

This means the memory required for cache: = **0.05 × 20 GB = 1 GB/day**

The memory size should also scale as our system grows.

Network and Bandwidth

Network / Bandwidth estimation tells us how much data flows in and out of our system per second.

Data Flow into Our System per Second (Ingress)

Incoming data in a day is ultimately saved in the storage.

- Given that **data stored in a day = 20 GB/day**

- Therefore, **incoming data in a day = 20 GB/day**

- Hence, **incoming data per second:**

$$20\text{GB}/(24 * 60 * 60) = \text{approx } 231\text{KB/sec}$$

Data Flow Out of Our System (Egress)

Outgoing data represents the data read or searched from our system.

- Given that we have **500 million search requests per day**
- Each search request returns **top 10 search results**
- Each search result is a tweet with an **average size of 200 bytes**
- Therefore, **outgoing data per day:**

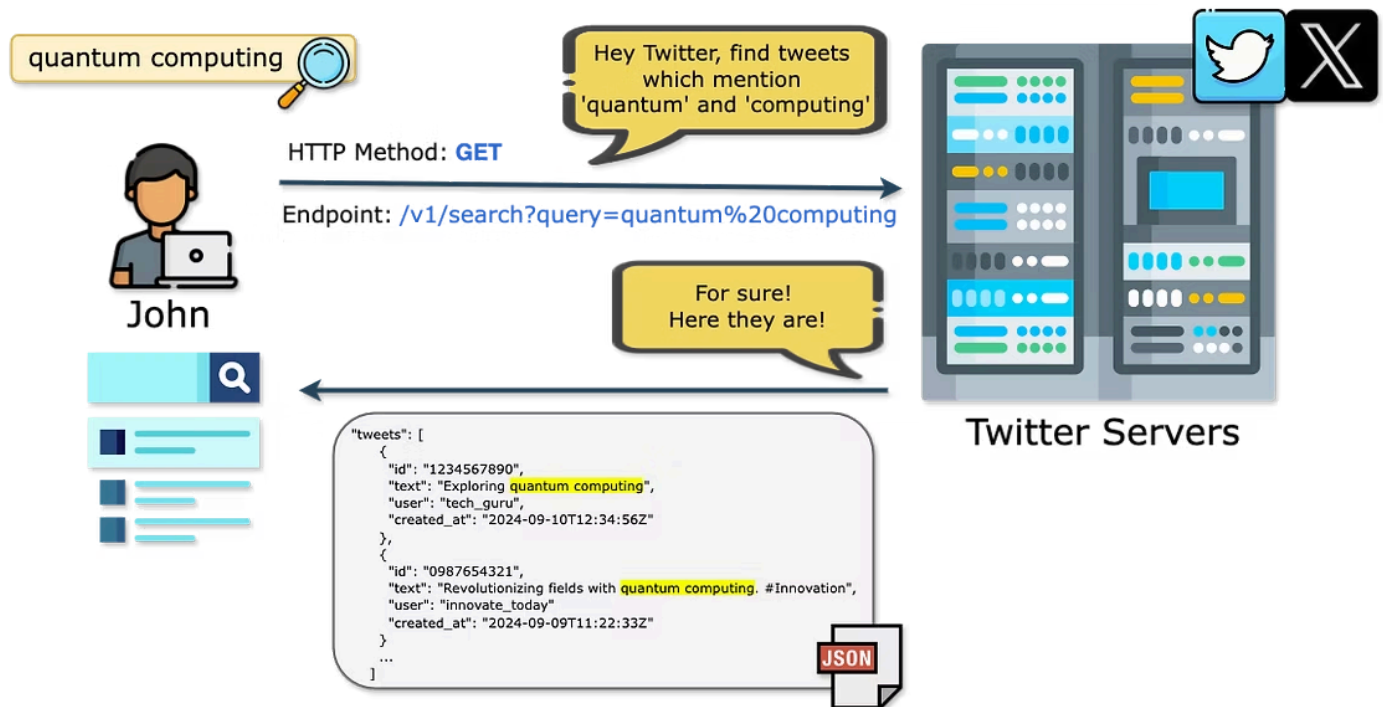
$$500\text{million} * 10 \text{ times} * 200\text{bytes} = 1,000 \text{ ,GB/day}$$

- Hence, **outgoing data per second:** = $1000 \text{ GB}/(24 * 60 * 60) = \text{approx } 11.5 \text{ MB/sec}$

API DESIGN

API DESIGN :Search

Let's understand what kind of API call goes to Twitter's servers when a user enters a keyword in the search bar and hits the search button.



HTTP Method

This tells the server what action to perform. Since we want to *get* a list of tweets related to our search keyword, we use the **GET** action.

Endpoint

This specifies where on the server to perform the action. For searching tweets, we use the **/v1/search** endpoint. We also pass our search keywords through this URL with the help of query parameters like this:

/v1/search?query=quantum%20computing

Now, you might wonder, where did this **%20** come from? Normally, you'd expect a space there between *quantum* and *computing*.

Imagine if we just sent a space in our URL like this:

/v1/search?query=quantum computing

This space could cause confusion, making it seem like our URL ends at *quantum*, and the rest of the part, *computing*, might be trimmed or ignored.

Encoding and Decoding

To avoid this confusion:

- We convert the space to **%20** in the URL.
 - This process is technically called **encoding**, where we convert special characters that might cause issues, like spaces, into a format that won't confuse the server.
 - The backend understands this encoding well and converts them back into their original characters, a process known as **decoding**.
-

HIGH LEVEL DESIGN

HIGH LEVEL DESIGN :Search Suboptimal

Let's say John goes to Twitter and searches for "quantum computing". What happens next looks something like this:

Step-by-Step Process

1. User Search Request

John opens Twitter and searches for "quantum computing". As soon as he does this, a search request is sent to the **API Gateway**.

2. Request Validation & Routing

The **API Gateway** validates this request and then passes it to the **Search Service** through a **Load Balancer**.

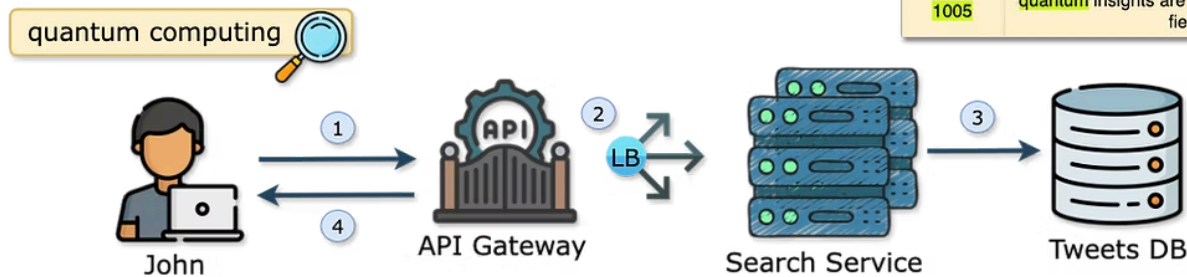
3. Keyword Extraction & Database Search

- The **Search Service** picks up the keywords, "**quantum**" and "**computing**", from John's query.
- It then searches the **Tweets Database** for all tweets that contain these two keywords.

4. Returning Search Results

Finally, all these tweets are returned to John as search results.

Tweet ID	Tweet
1001	"quantum leaps in tech are amazing! 🚀 #innovation"
1002	"quantum computing enhances cybersecurity. 🔒 #Security"
1003	"Attended a workshop on quantum effects; computing next!"
1004	"Exploring the amazing future of renewable energy! 🌍 #GreenTech"
1005	"quantum insights are transforming computing fields!"



Performance Issue

You might have guessed that this approach would work, but it is very **unoptimized**. It scans the entire table for every search, which can significantly slow down the search operation, especially when the system needs to support a **high volume of queries** like Twitter.

Let's now try to optimize this.

HIGH LEVEL DESIGN :Search Indexing

Previously, we saw that searching every tweet for certain keywords is slow and inefficient. How can we speed this up?

The Magic of Index DB

Imagine we have a magic list. Whenever we input a keyword into this list, it instantly gives us all the tweet IDs that contain this keyword. Essentially, there's no need to scan the entire table anymore.

This '**magic list**' is what we call an **Index DB**. When John searches for "*quantum computing*," the Index DB quickly shows which tweets contain these words, without checking every tweet.

How It Works: Step-by-Step

1. John initiates a search

- John opens Twitter and searches for "*quantum computing*."
- A search request is sent to the **API Gateway**.

2. Request validation and forwarding

- The **API Gateway** validates the request.
- It forwards the request to the **Search Service** via a **Load Balancer**.

3. Keyword search in Index DB

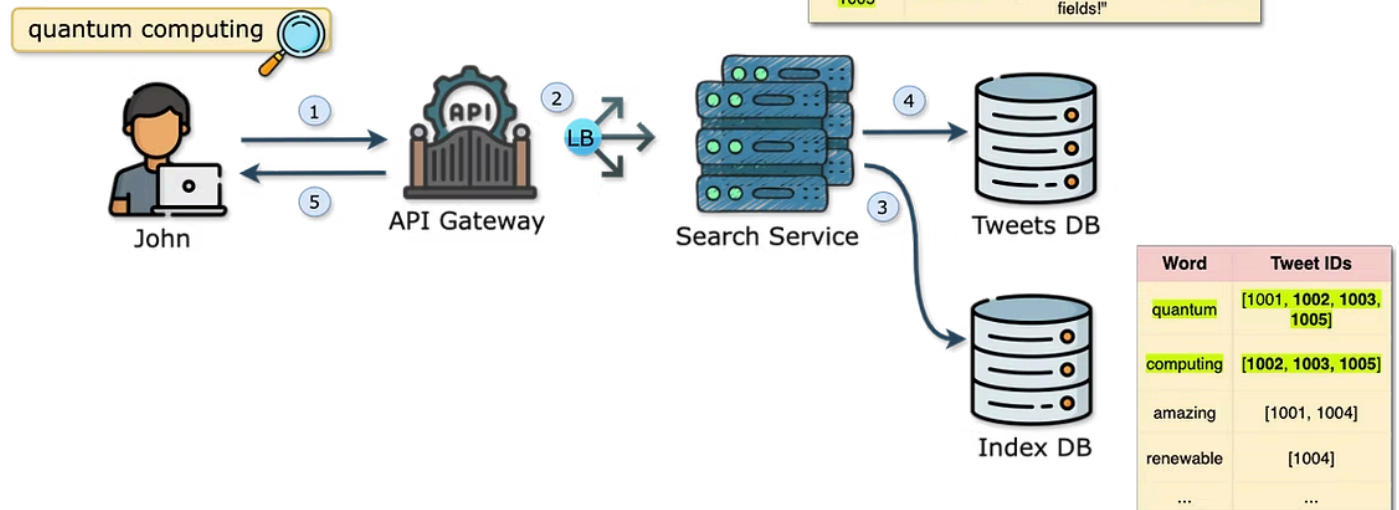
- The **Search Service** extracts keywords: "**quantum**" and "**computing**".
- It queries the **Index DB** to find tweet IDs containing these words:
 - "**quantum**" appears in tweets **1001, 1002, 1003, 1005**.
 - "**computing**" appears in tweets **1002, 1003, 1005**.
- The **intersection** of both lists is **1002, 1003, 1005** (tweets containing both words).

4. Retrieving the tweets

- The **Search Service** fetches these specific tweets from the **Tweets DB**.

5. Returning the results

- The tweets **1002, 1003, 1005** are sent back to John as search results.



With this optimized approach, searching becomes much faster, eliminating the need to scan every tweet in the database.

HIGH LEVEL DESIGN :How to support Fuzzy Search

"To err is human" – this common saying is very true. Often, we make mistakes while typing. But you might notice that Twitter still gives us the correct results.

This is what we call **Fuzzy Search**. *Fuzzy* literally means something that is not clear or is ambiguous.

Example

For example, suppose John searches for "**quantom computing**" instead of "**quantum computing**". He still receives results for "quantum computing."

How Does the Server Correct the Incorrect Word?

Let's understand from the diagram below how the server corrects the incorrect word. Most of the flow is the same except that steps **3 and 4** are added.

Step-by-Step Flow:

1. User Search Request:

- John opens Twitter and searches for "**quantum computing**".
- A search request is sent to the **API Gateway**.

2. API Gateway Processing:

- The **API Gateway** validates the request and forwards it to the **Search Service** via a **Load Balancer**.

3. Keyword Extraction & Initial Search:

- The **Search Service** picks up the keywords, '**quantom**' and '**computing**', from John's query.
- It checks if these keywords exist in the **Index DB**.
- It finds that there is no such word as '**quantom**' in the Index DB.

4. Query Correction:

- The **Search Service** calls the **Query Correction Service**.
- The **Query Correction Service** takes the incorrect word ('**quantom**') as input and returns the most likely correct word ('**quantum**').
- Consider this part of the high-level design as a black box. We will discuss its inner workings in the **Deep Dive Insights** section.

5. Search with Corrected Keywords:

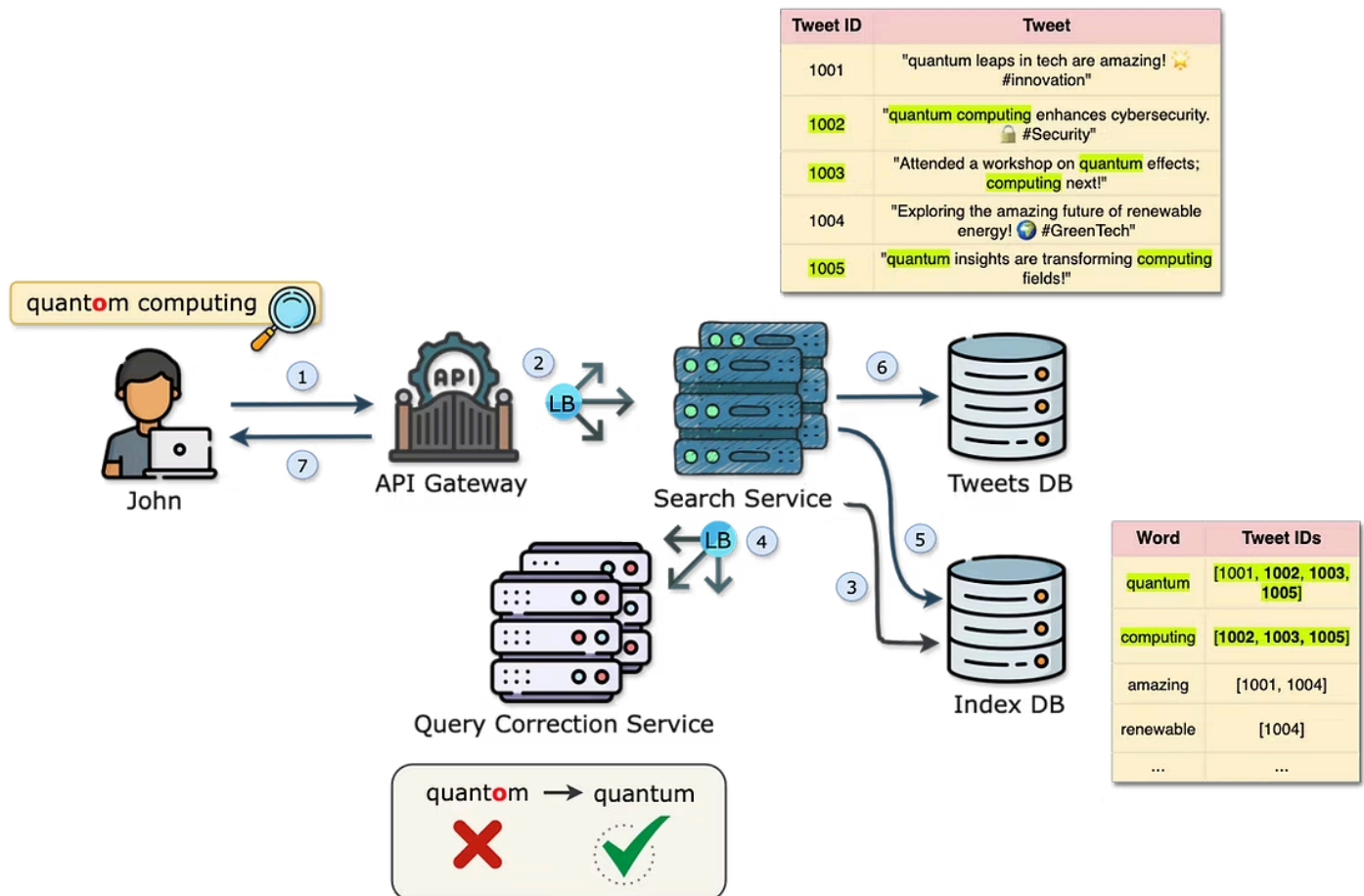
- The **Search Service** then searches the **Index DB** with the corrected keywords, '**quantum**' and '**computing**'.
- It finds that the keyword '**quantum**' appears in tweets **1001, 1002, 1003, 1005**, and '**computing**' in tweets **1002, 1003, 1005**.
- The **Search Service** takes the intersection of these, which is **1002, 1003, 1005**, because these tweets contain both '**quantum**' and '**computing**'.

6. Fetching Relevant Tweets:

- The **Search Service** retrieves these specific tweets from the **Tweets DB**.

7. Returning the Results:

- Finally, all these tweets (**1002, 1003, 1005**) are returned to John as search results.



This process ensures that even if users make spelling errors in their search queries, they still get relevant results using **Fuzzy Search**.

HIGH LEVEL DESIGN :Ranking

How Does Twitter Rank Search Results?

When John searches for "**quantum computing**," he receives a list of tweets. The most relevant tweet appears at the top of this list, with each subsequent tweet being slightly less relevant.

So, who does this sorting on the server side? **The Ranking Service**.

Let's understand this step-by-step. Most of the flow is the same as a standard search, but we've added one last step before returning the results, where we rank these tweets in order of relevance.

Step-by-Step Flow

1. User Initiates Search

- John opens Twitter and searches for "**quantum computing.**"
- A **search request** is sent to the **API Gateway**.

2. API Gateway Processing

- The **API Gateway** validates the request.
- It then forwards the request to the **Search Service** via a **Load Balancer**.

3. Keyword Processing

- The **Search Service** extracts keywords:
 - 'quantom' (misspelled)
 - 'computing'
- It checks the **Index DB** for these keywords.
- The **Index DB** finds no entry for 'quantom.'

4. Query Correction

- The **Search Service** calls the **Query Correction Service**.
- The **Query Correction Service** detects the incorrect word ('quantom') and corrects it to 'quantum.'

5. Searching for Relevant Tweets

- The **Search Service** searches the **Index DB** with the corrected keywords:
 - 'quantum' appears in **tweets 1001, 1002, 1003, 1005**.
 - 'computing' appears in **tweets 1002, 1003, 1005**.
- The **Search Service** takes the **intersection** of these results:
 - **Tweets 1002, 1003, and 1005** contain both 'quantum' and 'computing.'

6. Retrieving the Tweets

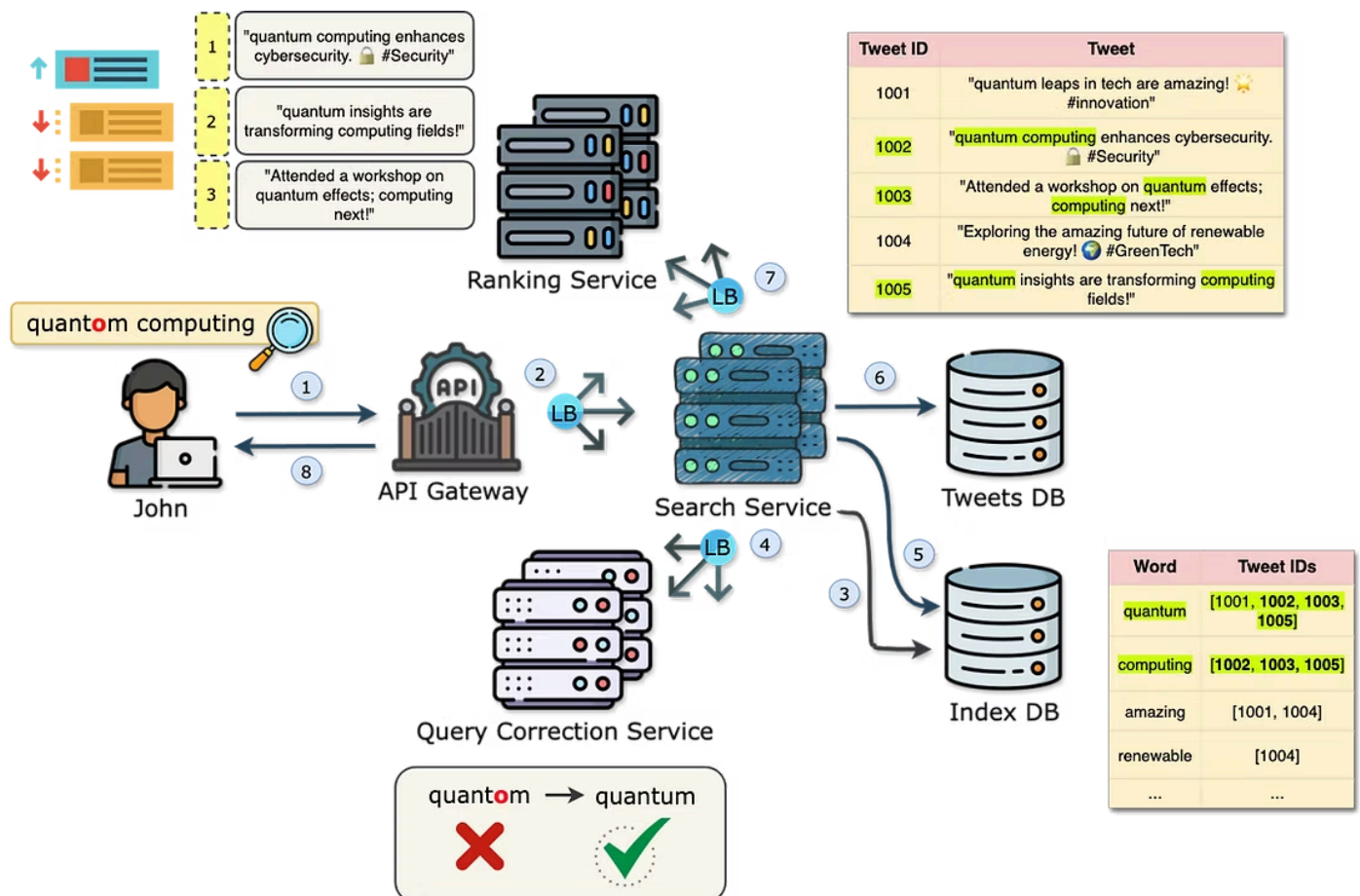
- The **Search Service** fetches these tweets from the **Tweets DB**.

7. Ranking the Results

- The retrieved tweets are sent to the **Ranking Service**.
- The **Ranking Service** orders them **from newest to oldest**.
- Instead of adding ranking logic to the **Search Service**, a separate **Ranking Service** was created.
 - This ensures **decoupling**, meaning if the ranking logic needs to change in the future, only the **Ranking Service** is updated.

8. Returning the Results

- Finally, the ranked tweets are returned to **John** as search results.



By structuring the system this way, Twitter ensures **efficient, accurate, and scalable** tweet searches while keeping components flexible for future improvements.

Let's take a look at the final design diagram for the search, including all the components — **Fuzzy Search**, **Ranking**, and **Caches** next to both databases to store frequently accessed data.

Search Process Flow

1. User Search Request

John opens Twitter and searches for "**quantum computing**." As soon as he does this, a search request is sent to the **API Gateway**.

2. API Gateway Validation

- The **API Gateway** validates this request.
- It then forwards the request to the **Search Service** via a **Load Balancer**.

3. Keyword Extraction and Initial Lookup

- The **Search Service** extracts the keywords: '**quantom**' and '**computing**' from John's query.
- It first **checks the cache** for these keywords.
- If **cache miss**, it checks the **Index DB**.
- The **Index DB** does not contain anything listed as '**quantom**'.

4. Query Correction Service

- The **Search Service** contacts the **Query Correction Service**.
- This service detects '**quantom**' as a likely typo and suggests the correct word: '**quantum**'.

5. Search with Corrected Keywords

- The **Search Service** searches the cache again with '**quantum**' and '**computing**'.
- If the cache does not contain them, it checks the **Index DB** and updates the cache (read-through mechanism).
- The **Index DB** lookup results:
 - '**quantum**' appears in **tweets 1001, 1002, 1003, 1005**.

- 'computing' appears in **tweets 1002, 1003, 1005**.
- The **intersection** of both results: **tweets 1002, 1003, 1005**.

6. Fetching Tweets Data

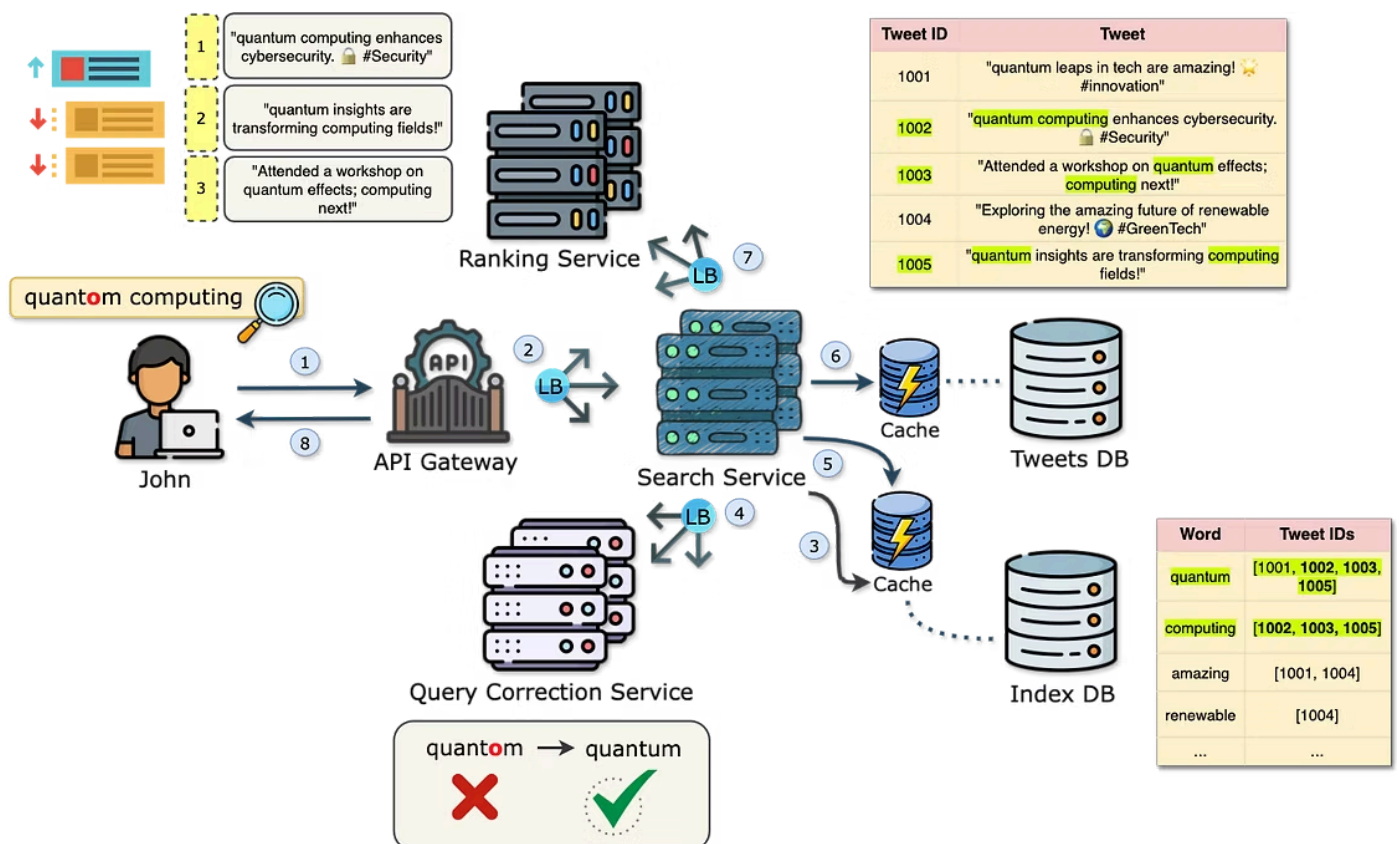
- The **Search Service** checks the **Tweets DB cache** for these tweet IDs.
- If the cache does not have them, it queries the **Tweets DB** and updates the cache (read-through mechanism).

7. Ranking the Results

- The **Search Service** sends the retrieved tweets to the **Ranking Service**.
- The **Ranking Service** orders them from **newest to oldest**.

8. Returning the Search Results

- The **Search Service** returns the ranked tweets to John as **search results**.



DEEP DIVE INSIGHTS: Database Selection

General Guidelines

The table below provides a high-level comparison of when to use **SQL** vs **NoSQL**, marked with ✓ (preferred) and ✗ (not preferred).

Criteria	SQL	NoSQL
Fast Data Access	✗	✓
Handles Large Scale	✗	✓
Fixed Structure Data	✓	✗
Flexible Structure Data	✗	✓
Complex Queries	✓	✗
Simple Queries	✗	✓
Frequent/Evolving Data Changes	✗	✓

Database Decision Table

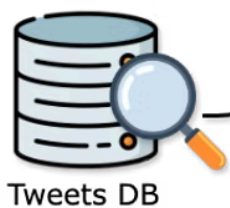
Database	Deciding Factors	Decision
Tweets DB	• High Scale - Handles search queries for millions of users daily, requiring efficient scalability. • Fast Access - Search results must load quickly; NoSQL supports low latency on high throughput. • Simple Query Pattern - Retrieves tweet (value) by tweet ID (key), a perfect use case for NoSQL.	NoSQL
Index DB	• High Scale - Handles search queries for millions of users daily, requiring efficient scalability.	NoSQL

Database	Deciding Factors	Decision
	• Fast Access - Needs to retrieve tweet IDs based on keyword searches quickly and efficiently. • Simple Query Pattern - Retrieves tweet IDs (value) by keyword (key), which fits well in a NoSQL key-value store.	

DEEP DIVE INSIGHTS: Data Modeling

Note: The tables shown in the high-level design were simplified versions just to illustrate the overall flow. This lesson provides a more detailed schema.

Tweets DB Schema



```
{
  "TweetId": "Unique identifier for each tweet",
  "Text": "Content of the tweet",
  "UserId": "Identifier for the user who posted the tweet",
  "CreatedAt": "Timestamp when the tweet was created"
}
```

- **Database Type:** NoSQL
- **Common Queries:** Find the tweet by tweet ID
- **Indexing:** Tweet ID

Note:

Because we have this common query to find the tweet by tweet ID, we create an index on the tweet ID. This sets a shortcut to quickly find the information by tweet ID.

Index DB Schema



Index DB

```
{  
  "Keyword": "A specific word or hashtag",  
  "TweetIds": ["List of tweet Ids having this keyword in the tweet"]  
}
```

- **Database Type:** NoSQL
- **Common Queries:** Grab all the tweet IDs for a keyword.
- **Indexing:** Keyword

Note:

Because we have this common query to grab all the tweet IDs for a keyword, we create an index on the keyword field. This sets a shortcut to quickly find the data by keyword.

DEEP DIVE INSIGHTS :Into Query Correction Service

We've seen how the Query Correction Service can turn John's misspelled or incorrect keyword into the correct one. Let's dive deeper into exactly how this happens.

Understanding with an Example

Imagine John types the incorrect keyword "**Fery**".

What do you think John actually meant? Here are some potential correct options:

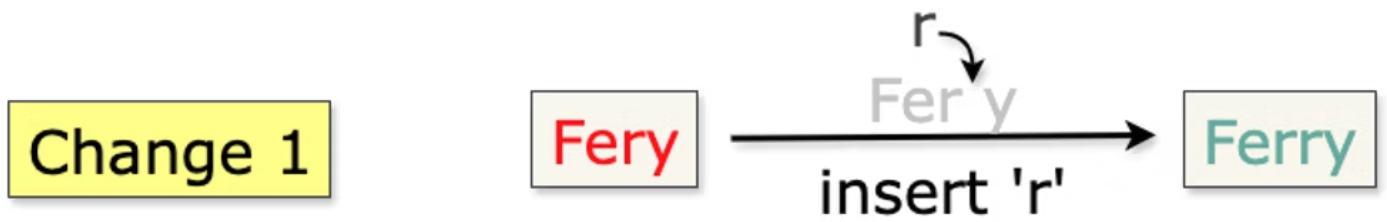
- **Ferry**
- **Fare**
- **Fair**

The most likely option is **Ferry** because it's the closest to what he typed. "*Closest*" here means requiring the fewest changes.

Transformation Steps

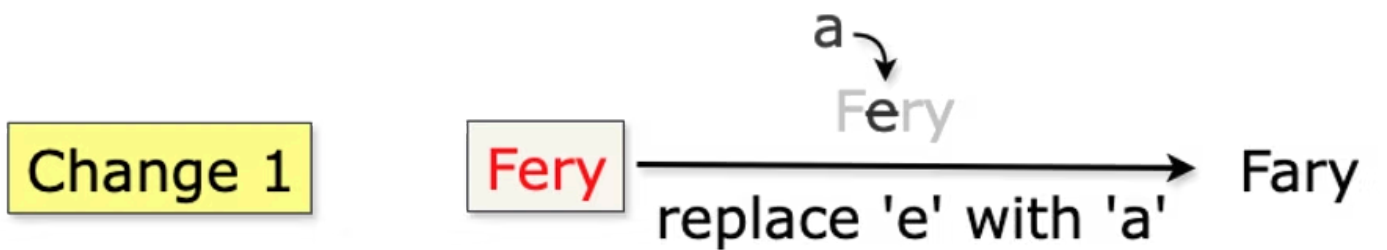
To turn **Fery** into **Ferry**, you only need to do **one change**:

- Add one 'r' → Fery → Ferry



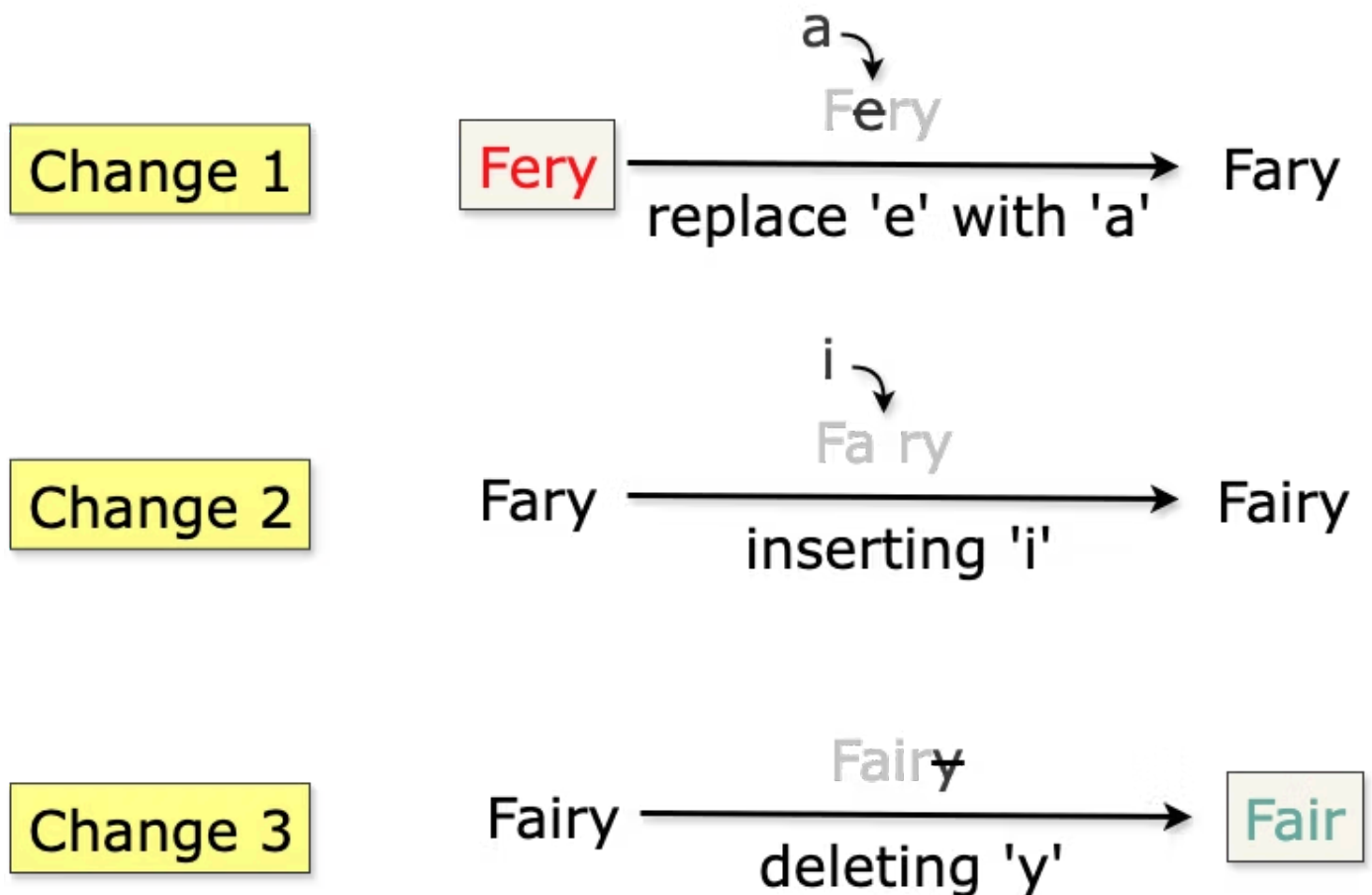
For **Fery** → **Fare**, it requires **two changes**:

1. Replace 'e' with 'a' → Fery → Fary
2. Replace 'y' with 'e' → Fary → Fare



For **Fery** → **Fair**, it requires **three changes**:

1. Replace 'e' with 'a' → Fery → Fary
2. Insert 'i' before 'r' → Fary → Fairy
3. Delete 'y' → Fairy → Fair



So, **Ferry** is the closest as it needs the fewest changes—just **one insertion**.

A change could be:

- Inserting a character
- Deleting a character
- Replacing a character

Levenshtein Distance

This count of changes (*aka closeness*) is **technically known as the Levenshtein Distance** between the two words.

The option with the **fewest changes (Ferry)** is typically selected, as it's considered the closest match to the intended word.

How the Query Correction Service Works

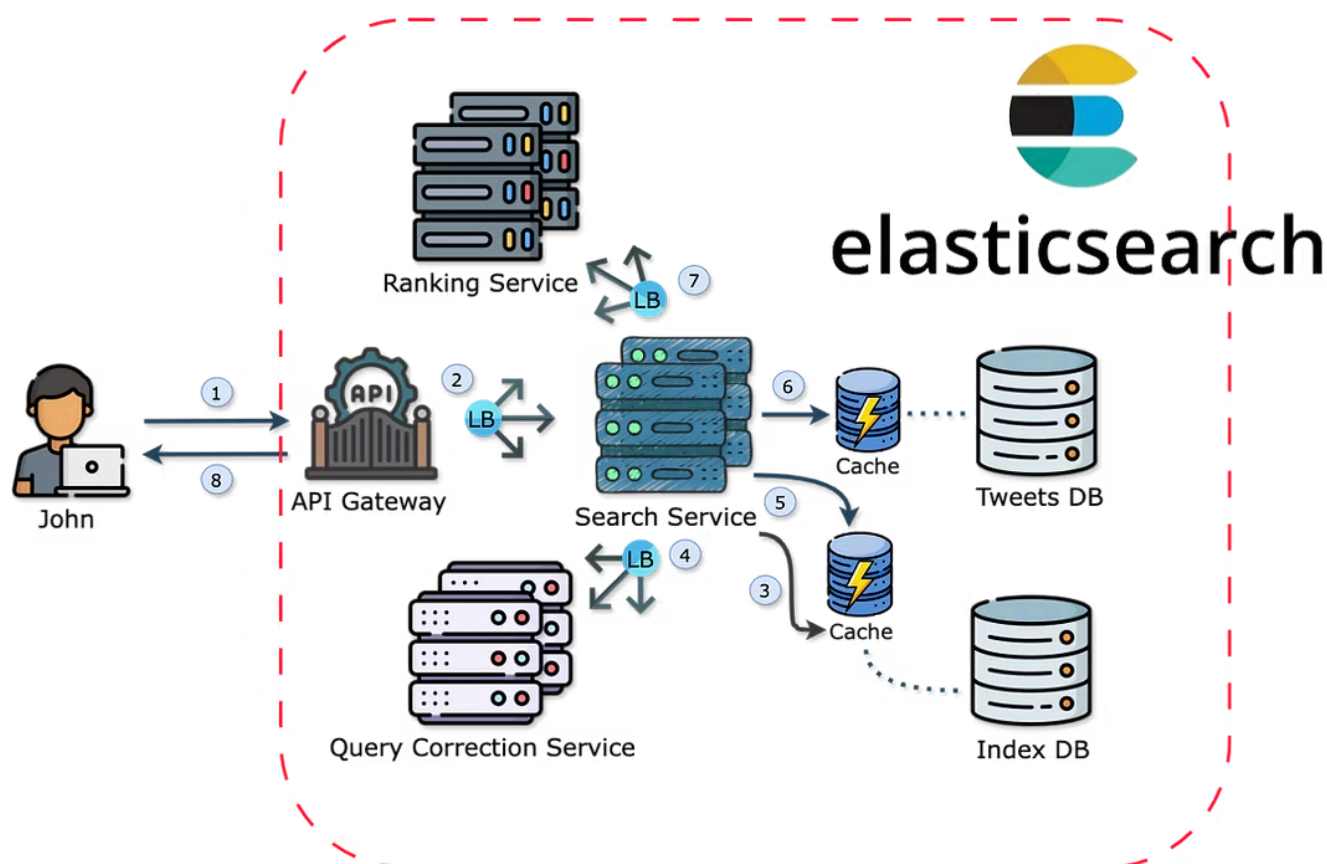
The Query Correction Service does something similar:

1. It calculates the **Levenshtein Distance** for all possible options from the incorrect keyword.
2. It chooses the one with the **minimum distance**.

This is the most likely option that John intended to type.

DEEP DIVE INSIGHTS :Elastic Search

Let's explore something interesting. The design we've created mirrors a popular and very powerful search tool known as **Elastic Search**.



What is Elastic Search?

Think of **Elastic Search** as a powerful search engine. It can handle vast amounts of data and delivers real-time search results. It provides search functionalities, including complex features like **fuzzy search**, and also helps **rank search results** based on their relevance.

Essentially, if we were to replace our entire design with **Elastic Search**, it would function in much the same way.

Who Uses Elastic Search?

Many popular websites, including **Twitter**, use **Elastic Search** to manage their vast data and provide **real-time search results**.